AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Elektrotechniki, Automatyki, Informatyki i Inżynierii Biomedycznej

Projekt dyplomowy

Enhancing Small Language Models via Chain-of-Thought Distillation
Ulepszanie małych modeli językowych poprzez destylację łańcucha myśli

Autor: *Kacper Rabczewski*
Kierunek studiów: *Informatyka i Systemy Inteligentne*
Opiekun pracy: *Dr inż. Krzysztof Kluza*

Kraków, 2026

Abstract

This thesis investigates whether symbolic chain-of-thought distillation (SCoTD) can measurably improve the reasoning accuracy of a small language model on text math problems. A strong teacher model generates structured reasoning traces (CoT) for GSM8K, which are cleaned and deduplicated, and then used to finetune a 3B-class student via parameter-efficient adapters (LoRA) under 4-bit quantization. The training stack relies on modern open tooling for reproducibility and efficient use of a single 24GB GPU. Evaluation uses strict numeric exact-match parsing on GSM8K.

Results show that SCoTD improves the student over a non-finetuned base with CoT prompting (78.54% vs 70.51%). A label-only variant also improves over its baseline (15.01% vs 10.99%), though strict formatting requirements likely undercount true ability. The teacher reaches 93.61% on the GSM8K test split. Contributions include a cost-aware CoT data generation pipeline, a reproducible finetuning setup (LoRA + 4-bit nf4 + bf16), and an empirical analysis of accuracy, training dynamics, and operational costs. These findings support SCoTD as a practical method to enhance small models' reasoning under resource constraints.

Streszczenie

Niniejsza praca bada, czy symboliczna destylacja łańcucha myśli (SCoTD) może wymiernie poprawić dokładność rozumowania małego modelu językowego w tekstowych zadaniach matematycznych. Silny model nauczyciela generuje ustrukturyzowane ślady rozumowania (CoT) dla zbioru GSM8K, które są czyszczone i deduplikowane, a następnie wykorzystywane do dostrojenia modelu ucznia klasy 3B przy użyciu efektywnych parametrowo adapterów (LoRA) z 4-bitową kwantyzacją. Stos technologiczny opiera się na nowoczesnych otwartych narzędziach zapewniających powtarzalność i efektywne wykorzystanie pojedynczego GPU 24 GB. Ewaluacja wykorzystuje ścisłe dopasowanie numeryczne na zbiorze GSM8K.

Wyniki pokazują, że SCoTD poprawia wyniki ucznia w porównaniu do modelu bazowego bez dostrajania z promptowaniem CoT (78,54% vs 70,51%). Wariant uczony tylko na etykietach również poprawia wyniki względem swojego poziomu odniesienia (15,01% vs 10,99%), chociaż rygorystyczne wymagania formatowania prawdopodobnie zaniżają rzeczywistą zdolność. Nauczyciel osiąga 93,61% na zbiorze testowym GSM8K. Wkład pracy obejmuje potok generowania danych CoT uwzględniający koszty, powtarzalną konfigurację dostrajania (LoRA + 4-bit nf4 + bf16) oraz empiryczną analizę dokładności, dynamiki uczenia i kosztów operacyjnych. Wyniki te wspierają SCoTD jako praktyczną metodę ulepszania rozumowania małych modeli przy ograniczonych zasobach.

Keywords: LLM, chain-of-thought, distillation, LoRA, QLoRA, GSM8K, reasoning, quantization

Contents

1. Introduction 7

2. Background 11

2.1. Large language models 11

2.1.1. Objective and pretraining 11

2.1.2. Transformer 11

2.1.3. Decoder-only LLMs 13

2.1.4. Scaling laws 13

2.1.5. Limitations and small-model challenges 13

2.2. Prompting 14

2.2.1. Paradigms 14

2.3. Knowledge distillation 14

2.3.1. Objectives and loss design 14

2.3.2. Distillation for sequence models and LLMs 15

2.3.3. Distilling reasoning 15

2.3.4. When to expect gains 16

2.4. Parameter-efficient fine-tuning (PEFT) 16

2.4.1. Method families 16

2.4.2. Quantization-aware PEFT 17

2.5. Tokenizers 17

2.5.1. Subword methods 17

2.5.2. Special tokens and sequence framing 19

2.6. GSM8K 20

2.6.1. Dataset structure 20

3. Methodology 23

3.1. System requirements 23

3.1.1. Functional requirements 23

3.1.2. Non-functional requirements 23

3.1.3. Constraints 24

3.1.4. Success criteria ... 24
3.1.5. Risks and mitigation ... 24
3.2. Architecture ... 24
3.2.1. Components and data flow ... 24
3.3. Teacher CoT dataset generation ... 25
3.3.1. Subset selection and async generation ... 25
3.3.2. Formatting and correctness ... 26
3.3.3. Persistance and progress monitoring ... 26
3.4. Cleaning ... 26
3.5. Preprocessing ... 26
3.6. Prompt templates ... 26
3.7. Training ... 28
3.7.1. Model configuration and data ... 28
3.7.2. Training procedure ... 28
3.7.3. Export ... 28
3.8. Benchmarking ... 28
3.8.1. Evaluation pipeline ... 28
3.8.2. Scoring ... 29
3.9. Environment setup ... 29
4. Results ... 31
4.1. Evaluation setup recap ... 31
4.2. Checkpointing ... 32
4.3. Training ... 32
4.3.1. Label-only model ... 32
4.3.2. SCoTD model ... 33
4.3.3. Comparison ... 33
4.4. Output length and latency distributions ... 35
4.5. Dataset filtering outcomes ... 36
4.6. Cost analysis ... 36
5. Discussion ... 37
5.1. Interpretation of quantitative outcomes ... 37
5.2. Role of dataset filtering ... 37
5.3. Training dynamics and generalization ... 37
5.4. Formatting sensitivity ... 38
5.5. Cost and efficiency implications ... 38

5.6. Limitations and threats to validity 38
5.7. Comparison to literature 38
5.8. Implications and future directions 39
6. Conclusion 41
6.1. Future work 42

1. Introduction

Large language models (LLMs) have achieved strong performance on a wide range of language tasks. However, small models (sub-4B parameters) often fall short on multi-step reasoning benchmarks such as GSM8K, where solutions require decomposing a problem into intermediate steps and computing a precise numeric answer [1]. Chain-of-thought (CoT) prompting can elicit step-by-step reasoning from strong teachers and improve accuracy [2], especially when combined with self-consistency sampling [3]. Yet, relying on large models at inference time is costly and slow. This motivates distilling teacher reasoning traces into a small student that can run efficiently on commodity hardware.

This thesis studies symbolic chain-of-thought distillation (SCoTD): a strong teacher generates structured CoTs for GSM8K questions; a small student is finetuned to reproduce these traces and final answers. The student is trained with parameter-efficient fine-tuning (LoRA) [4] on quantized weights (QLoRA) [5], enabling training on a single 24 GB GPU while preserving headroom to learn reasoning behavior. The approach uses standard open-source tooling for models and tokenization [6, 7, 8].

Problem and motivation

Small models are attractive due to their lower latency, footprint, and cost, but they struggle with multi-step math reasoning under greedy decoding. CoT distillation transfers the teacher's intermediate reasoning patterns, potentially improving student accuracy without increasing inference-time cost. The practical question is whether a 3B-class model can benefit sufficiently from SCoTD to close part of the gap to teacher performance on GSM8K.

Research questions

To systematically evaluate the effectiveness of the proposed distillation method and understand its impact relative to baselines, this study addresses the following specific research questions:

- RQ1: Does SCoTD improve a 3B-class model on GSM8K compared to the non-finetuned base under CoT prompting?
- RQ2: How does label-only supervised finetuning compare to SCoTD and base baselines under strict numeric exact-match evaluation?

Scope

The study focuses on a single small decoder-only model (`Qwen/Qwen2.5-3B`) [9] and a single dataset (GSM8K) [1]. Teacher (DeepSeek-R1 Distill [10]) CoTs are produced via an API-accessible, strong reasoning model configured to emit structured steps and a single-line final answer. The student is trained with LoRA on 4-bit quantized base weights using bfloat16 precision, gradient checkpointing, and standard optimizers. Evaluation uses greedy decoding with strict numeric parsing; outputs that deviate from the expected format are counted as incorrect.

Contributions

This thesis makes several practical and empirical contributions to the field of efficient language model reasoning. Specifically, the work delivers:

– A practical, cost-aware pipeline for generating and cleaning teacher CoTs for GSM8K, with concurrency-tuned API calls and per-sample metadata for analysis.

– A reproducible PEFT training setup (LoRA + 4-bit nf4 + bfloat16) fitting on a single 24 GB GPU while targeting attention and MLP projection modules.

– Benchmark giving empirical evidence that SCoTD improves a 3B-class model over base CoT prompting on GSM8K (78.54% vs 70.51%), with a strong teacher at 93.61% and a label-only student that improves over baseline (15.01% vs 10.99%) despite strict-format penalties.

– Operational notes on prompt design, cleaning/deduplication, truncation policy, and evaluation pitfalls that affect measured accuracy.

Method overview

The pipeline comprises teacher CoT generation, cleaning and deduplication, dataset processing, student finetuning (SCoTD and label-only variants), and evaluation. The student relies on PEFT with LoRA adapters [4] over a quantized backbone [5]. Tokenization is handled with a fast subword tokenizer [7, 8]. Inference uses greedy decoding and a strict numeric exact-match metric. The software stack builds on Transformers and related tooling [6]. The teacher is a strong reasoning model accessed via OpenRouter [11].

Results summary

SCoTD yields a clear gain over the base model under CoT prompting (78.54% vs 70.51%). Label-only fine-tuning improves over its baseline (15.01% vs 10.99%), though strict formatting likely undercounts correct numeric computations. The teacher achieves 93.61% on GSM8K test, providing headroom and confirming the value of distilling high-quality reasoning traces.

Author background and motivation

The author is a Co-Founder at an AI phishing detection startup that uses LLMs to analyze URLs and webpage content for malicious indicators, and a Machine Learning Engineer at OneRail building LLM-powered support workflows for same-day logistics. These roles highlighted the need for reliable, cost-efficient reasoning under tight latency and single-GPU constraints, motivating SCoTD as a practical path to higher accuracy under strict exact-match parsing.

2. Background

This chapter establishes the theoretical foundations required to understand the proposed solution. It reviews the architecture of Large Language Models, specifically the Transformer and decoder-only variants, and discusses the mechanisms of prompting, knowledge distillation, and parameter-efficient fine-tuning. Additionally, it introduces the specific tools and benchmarks, such as tokenizers and the GSM8K dataset, that are central to the experiments.

2.1. Large language models

Large language models (LLMs) lie at the core of the current AI revolution. They are parametric probabilistic models that learn to represent the distribution of natural language sequences and generate text by autoregressively predicting one token at a time. Formally, given a tokenized sequence $(x_1, \ldots, x_T)$, an LLM models $p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$ and is trained to minimize the cross-entropy between predicted and observed tokens. LLMs derive their broad linguistic and factual knowledge from pretraining on massive text corpora using this objective, which at scale enables emergent capabilities such as in-context learning and reasoning.

2.1.1. Objective and pretraining

The standard objective is a next-token prediction:

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}),$$

optimized over large, diverse datasets using gradient descent. Conceptually, this process can be understood as a vast, GPU-driven lossy compression—the model distills the statistical regularities and patterns of human language into a compact, parameterized form that enables general-purpose text understanding and generation. While pretraining is purely self-supervised, downstream adaptation may use supervised fine-tuning or preference-based methods to align behavior.

2.1.2. Transformer

The Transformer architecture [12] underlies most modern LLMs. It consists of stacked blocks that process sequences through attention and feed-forward layers. The main building blocks are:

– **Embeddings and positional encoding**: Tokens are mapped to dense vectors and augmented with position information, either sinusoidal or learned, to make the model order-aware.

– **Multi-head self-attention**: For each position, queries Q, keys K, and values V are computed by linear projections of the hidden states. Scaled dot-product attention aggregates information across positions:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where d_k denotes the dimensionality of the query and key vectors. Multiple heads attend in parallel to different subspaces, and their outputs are concatenated and projected.

– **Position-wise feed-forward network (FFN)**: A two-layer Multi-Layer Perceptron (MLP) with a nonlinearity such as GELU (Gaussian Error Linear Unit) is applied independently to each position.

– **Residual connections and layer normalization**: Each sublayer is wrapped with residual additions and normalization for stability.

Computationally, attention has $O(L^2 d)$ time and $O(L^2)$ memory in sequence length L (per batch and head factors omitted), while FFNs dominate parameter count with $O(d^2)$. The architecture of a standard Transformer block is illustrated in Figure 2.1.

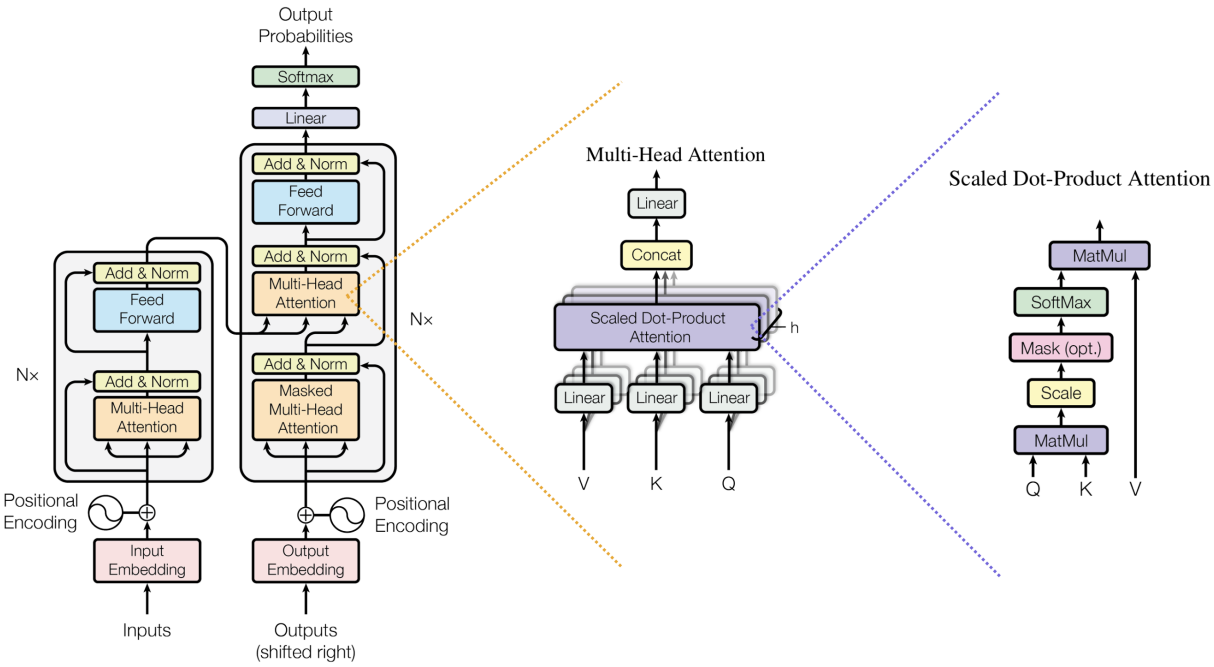


Figure 2.1. Transformer encoder-decoder block [12].

2.1.3. Decoder-only LLMs

Decoder-only LMs use a lower-triangular mask in self-attention so that token t attends only to positions $< t$. This enforces autoregressive factorization and enables left-to-right generation. During inference, key-value (KV) caching stores past projections to avoid recomputation, reducing per-token latency from quadratic to approximately linear in L for attention, with constant-depth projections per layer.

2.1.4. Scaling laws

Transformers scale predictably with model size (parameters), data (tokens), and compute. Empirically, test loss follows power-law scaling with these resources [13], implying steady gains as capacity and data increase. Practical costs rise superlinearly with size due to:

– **Training**: attention's $O(L^2)$ complexity amplifies FLOPs and memory.

– **Inference**: latency and memory scale with depth and width; context length increases multiply attention cost without specialized kernels.

Despite costs, scaling yields emergent capabilities such as in-context learning and improved generalization [14]. These relationships are visualized in Figure 2.2, showing the power-law trends.

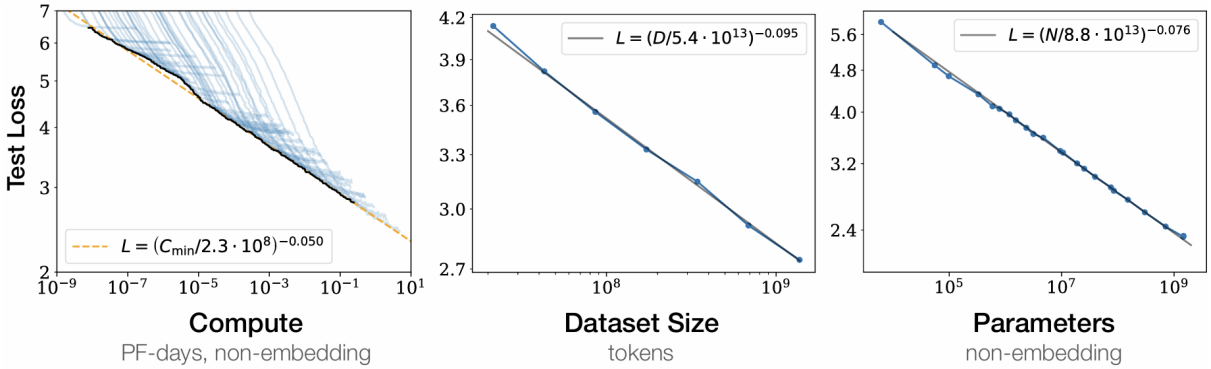


Figure 2.2. Scaling laws: test loss vs. effective compute/model size/data on logarithmic axes [13].

2.1.5. Limitations and small-model challenges

Large language models are known to have several limitations including:

– **Hallucinations**: the model may produce fluent but unsupported statements.

– **Context and computation limits**: finite context windows and lack of external working memory hinder long-horizon tasks.

– **Compositionality**: chaining steps and maintaining intermediate variables is difficult under greedy decoding.

Small models (e.g., <4B parameters) face additional constraints which degrade multi-step arithmetic and logical reasoning, especially when strict output formats are required. While prompting and decoding strategies can mitigate some issues, structural capacity often remains the bottleneck without additional supervision or architectural support.

2.2. Prompting

Prompting is a practice of specifying a model's task through natural language. It functions as a programming interface to language models. Instruction tuning and alignment methods improve adherence to prompts [15].

2.2.1. Paradigms

Several prompting paradigms have emerged, often combined in practice:

- **Zero-shot prompting**: The model receives only task instructions and the input. This tests inherent generalization from pretraining and any instruction tuning. It is simple and cheap but sensitive to wording and often weaker on tasks requiring intermediate computation.
- **Few-shot prompting**: The prompt includes k input-output examples that implicitly define the task and desired format, enabling in-context learning [14].
- **Chain-of-Thought (CoT) prompting**: The prompt requests step-by-step reasoning, eliciting intermediate rationales before the final answer [2]. CoT can improve complex, multi-step tasks like arithmetic by encouraging deliberate decomposition, but increases latency and token usage.

2.3. Knowledge distillation

Knowledge distillation (KD) transfers the behavior of a high-capacity teacher model to a typically smaller student [16]. The core idea is that softened target distributions carry "dark knowledge" about class or token similarities that hard labels do not expose, often improving generalization while enabling compression and faster inference.

2.3.1. Objectives and loss design

Response-based KD is the most widely used approach. In this setting, the student is trained to match the teacher's output distribution. Let $z^{(T)}$ and $z^{(S)}$ be teacher and student logits (the raw, unnormalized scores output by the network before the softmax function), and $T > 0$ a temperature. The softened distributions are $p^{(T)} = \mathrm{softmax}(z^{(T)}/T)$ and $p^{(S)} = \mathrm{softmax}(z^{(S)}/T)$. A common loss combines distillation with standard supervision:

$$\mathcal{L} = (1-\alpha)\,\mathrm{CE}(y,\,\mathrm{softmax}(z^{(S)})) + \alpha\,T^2\,\mathrm{KL}\Big(p^{(T)} \,\|\, p^{(S)}\Big)\,,$$

where y are hard labels, CE denotes cross-entropy loss, KL is the Kullback–Leibler divergence, and $\alpha \in [0, 1]$ balances the two terms. The cross-entropy term uses untempered softmax (equivalent to $T = 1$) because it targets the ground-truth labels directly. The T^2 factor compensates for gradient scaling under temperature [16]. When $\alpha = 0$, the loss reduces to standard supervised learning on hard labels; when $\alpha = 1$, the student learns purely from the teacher's soft predictions without ground-truth supervision.

Beyond responses, feature-based KD aligns intermediate representations, while relation-based KD matches instance relations. Self-distillation trains a student from its own or a previous generation's predictions (Born-Again Networks [17]); data-augmented KD uses pseudo-labels on unlabeled data (Noisy Student [18]). A comprehensive survey by Gou et al. [19] reviews these and other distillation paradigms, taxonomizing methods by knowledge type (response, feature, relation) and training scheme (offline, online, self-distillation), and discussing applications across vision, NLP, and other domains.

2.3.2. Distillation for sequence models and LLMs

In language modeling, KD is applied at either:

– **Token level**: match teacher logits or probabilities at each decoding step, often alongside cross-entropy to the ground truth.

– **Sequence level**: train on sequences decoded by the teacher (e.g., beam search), simplifying the target distribution and reducing mode entropy [20].

Both offline (fixed teacher) and online (co-trained or periodically refreshed teacher) regimes exist. For autoregressive models, exposure bias remains; KD reduces variance but does not remove the mismatch between training and generation. Practical choices include which layers to distill, how to weight losses over time, and whether to distill only on high-confidence tokens.

2.3.3. Distilling reasoning

Symbolic Chain-of-Thought Distillation (SCoTD) transfers step-by-step reasoning from a large teacher to a smaller student by training on teacher-generated rationales rather than logits [21]. Concretely: (i) curate a few-shot CoT prompt set; (ii) for each unlabeled input x, sample multiple CoT rationales z and a prediction y from the teacher; (iii) optionally filter samples to keep only label-correct pairs when gold labels are available; and (iv) fine-tune the student with the standard token-by-token language modeling objective to generate the concatenated rationale and answer. At inference, the student can decode a single rationale+answer greedily or sample multiple reasoning paths and select the majority answer via self-consistency [3].

Empirically, SCoTD enables 125M-1.3B students to "think step-by-step"and improves accuracy on commonsense QA benchmarks (CSQA, QuaRel, OpenBookQA). A key finding is that oversampling rationales per input (e.g., $\sim$30 CoTs/example) is more important than aggressive filtering. Self-consistency particularly helps when training used noisy, unfiltered CoTs (few-shot setting), whereas its benefit diminishes once supervised filtering removes incorrect teacher answers.

2.3.4. When to expect gains

KD is most effective when the teacher's distribution contains informative structure that the student cannot infer from hard labels alone. For CoT, expected gains increase with teacher rationale quality and student capacity to model longer sequences.

2.4. Parameter-efficient fine-tuning (PEFT)

Full fine-tuning updates all model parameters and optimizer states, which scales memory, compute, and communication roughly linearly with parameter count and often requires multi-GPU setups. Optimizers like Adam maintain first and second moments, tripling memory relative to parameters alone. Checkpointing, gradient storage, and activation memory further increase requirements. Parameter-efficient fine-tuning (PEFT) alleviates these costs by freezing the backbone and training a small number of additional parameters that modulate the forward pass. This also improves modularity (task-specific heads), reduces overfitting, and enables fast switching between tasks.

2.4.1. Method families

Several PEFT families differ in where trainable parameters are introduced and how they influence the network:

- **Adapters**: Small bottleneck modules (down-projection, nonlinearity, up-projection) are inserted within Transformer blocks and trained while the base model is frozen [22]. They achieve near full fine-tuning performance with orders of magnitude fewer trainable parameters and support composition across tasks.

- **Prompt and prefix tuning**: Instead of modifying internal weights, these methods learn task-specific soft prompts. Prompt tuning optimizes a small set of embeddings prepended to the input [23], while prefix-tuning optimizes continuous key/value prefixes injected into attention at each layer [24]. These approaches are extremely parameter-efficient and work well when the pre-trained model already encodes the needed capabilities; performance can depend on model scale and task complexity.

- **Low-rank adaptation (LoRA)**: LoRA freezes the original weight matrix W and parameterizes the update as a low-rank decomposition $\Delta W = AB$ with rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$ [4]. A common scaled form is

$$W' = W + \frac{\alpha}{r} AB,$$

  where $A \in \mathbb{R}^{d_{\text{out}} \times r}$ and $B \in \mathbb{R}^{r \times d_{\text{in}}}$ are the only trainable parameters and α controls the update magnitude. LoRA is typically applied to attention and MLP projection matrices. Benefits include: (i) minimal trainable parameters, (ii) negligible inference overhead when merged into W, and (iii) modular adapters that can be stored and shared independently of the base model.

2.4.2. Quantization-aware PEFT

Quantization-aware fine-tuning combines PEFT with low-precision base weights to further reduce memory. QLoRA keeps the frozen backbone in 4-bit quantization (e.g., NF4) with per-channel scaling and "double quantization" to compress scales, while performing forward/backward compute in higher precision (e.g., bf16) and training only the PEFT parameters [5]. In this setup:

- The quantized backbone reduces memory footprint without updating full-precision weights.
- LoRA (or other PEFT parameters) are maintained in higher precision and receive gradients.
- Careful quantizer design (NF4) and optimizer choices preserve training stability and accuracy close to full-precision baselines.

This approach enables fine-tuning larger backbones under tight memory budgets and decouples adaptation capacity (set by PEFT parameters like LoRA rank) from the storage format of the base model. Practical backends for 4-bit inference/training are widely available [25]. Figure 2.3 depicts the memory layout of this method, highlighting the separation between frozen quantized weights and trainable adapters.

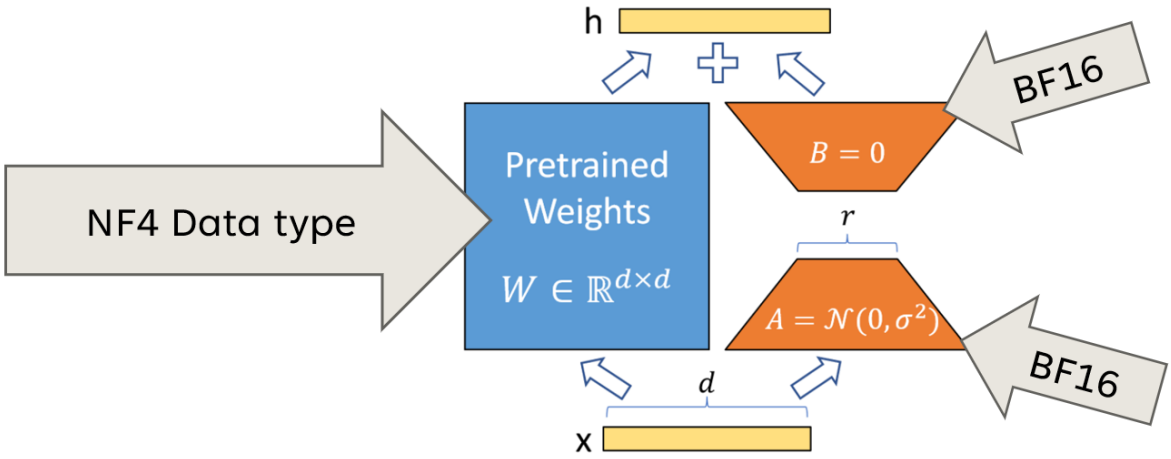


Figure 2.3. Model parameters under QLoRA NF4 + LoRA: frozen backbone in 4-bit quantization with per-channel scales; trainable LoRA parameters in higher precision. Forward/backward compute in bf16 [26].

2.5. Tokenizers

Tokenization converts raw text into a sequence of discrete token IDs drawn from a fixed vocabulary. Language models consume token IDs and map them to embedding vectors.

2.5.1. Subword methods

Pure word-level vocabularies suffer from out-of-vocabulary (OOV) issues, while character-level tokenization produces very long sequences. Subword tokenization offers a compromise by decomposing words into frequent subunits that compose to any string, including rare words and misspellings [8].

Common approaches:

– **Byte-Pair Encoding (BPE)**: the most widely used method in modern LLMs, starts from a large text corpus and iteratively merges the most frequent adjacent pairs to form larger subwords until a target vocabulary size is reached [8]. BPE yields deterministic segmentations given the learned merges and has no explicit probabilistic model.

– **WordPiece**: subword units to maximize likelihood of a training corpus, typically under a simple probabilistic model over segmentations; greedy decoding selects the best segmentation. It typically includes reserved continuation markers to indicate that a subword is not word-initial.

– **SentencePiece**: optimizes a probabilistic unigram model over a candidate subword inventory and prunes units that hurt likelihood [7]. It supports sampling alternative segmentations during training for robustness and can operate directly on raw text without external pre-tokenization.

Implementation frameworks such as SentencePiece and Transformers provide standardized training, encoding, and decoding pipelines [7, 6]. Variants like byte-level BPE ensure coverage of arbitrary Unicode by operating on UTF-8 bytes, which removes true OOVs at the cost of sometimes longer sequences for non-Latin scripts. An example of how text is split into tokens is shown in Figure 2.4.

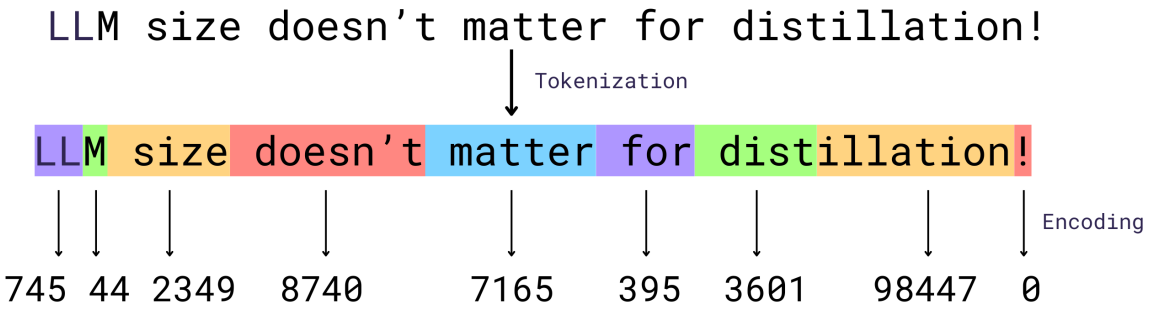


Figure 2.4. Tokenization example for OpenAI GPT-4o Tokenizer. The colored segments show individual tokens, and the numbers below represent the unique integer indices assigned to each token in the vocabulary.

2.5.1.1. Vocabulary

Larger token vocabularies can represent up to whole words, reducing sequence length and attention cost. However, they implicitly increase embedding and output layer sizes, memory, and may cause overfitting. Smaller vocabularies generalize better to rare forms but inflate sequence length, which increases quadratic attention cost and risk of truncation. Practical choices range from 16k-200k subwords.

2.5.1.2. Normalization and pre-tokenization

Many pipelines normalize text (e.g., Unicode NFKC, lowercasing, whitespace handling) to improve robustness; SentencePiece can integrate normalization and avoid external word boundary rules [7]. Pre-tokenizers may split on whitespace and punctuation to enforce word boundary awareness before applying subword merges. Choices here affect how punctuation, casing, and whitespace survive round-trip detokenization.

2.5.2. Special tokens and sequence framing

Special tokens mark structure and control decoding:

– **BOS/EOS**: Begin-of-sequence and end-of-sequence delimit the sequence and signal termination.

– **PAD**: Padding aligns sequences within a batch to a common length; attention masks ensure pads are ignored during loss computation and attention.

– **Instruction tokens**: Special markers that indicate task boundaries or specify desired behavior (e.g., `<|user|>`, `<|assistant|>`, `<|thought|>`).

Figure 2.5 demonstrates how these tokens structure a conversation within the model's context window.

```
<|bos|>
<|user|>
You are given a list of numbers: [3, 1, 4, 1, 5].
Please compute their sum and provide the result.
<|assistant|>
The sum of [3, 1, 4, 1, 5] is 14.
<|eos|>
```

Figure 2.5. Example of conversation-formatted instruction-response prompt.

2.6. GSM8K

GSM8K is a benchmark of 8.5K English grade-school math word problems designed to evaluate multi-step reasoning in natural language [1]. Problems typically require 2-8 steps of elementary arithmetic ($+, -, \times, \div$) and include human-written, natural-language solutions with inline "calculator annotations" (intermediate computations) and a final integer answer on a separate line. The benchmark is widely used to study prompting and reasoning supervision; standard scoring is exact numeric match, often evaluated under chain-of-thought prompting and self-consistency sampling [2, 3].

2.6.1. Dataset structure

Released in English via HuggingFace Datasets [27], GSM8K provides two configurations: a main set (question, rationale, final answer) and a socratic variant that interleaves sub-questions within the solution. Official splits comprise 7,473 training and 1,319 test problems [1]; each instance pairs a question string with a multi-step natural-language solution containing intermediate calculations and a single integer final answer. A typical sample from the dataset is presented in Figure 2.6, displaying the question and the solution consisting of step-by-step rationale and final numeric answer.

Problem

```
Natalia sold clips to 48 of her friends in April, and then she sold half as
many clips in May. How many clips did Natalia sell altogether in April and
May?
```

Solution

```
Natalia sold 48/2 = <<48/2=24>>24 clips in May. Natalia sold 48+24 =
<<48+24=72>>72 clips altogether in April and May. #### 72
```

Figure 2.6. GSM8K example: question, rationale with intermediate steps, and final answer.

This chapter has provided the necessary background on language modeling, distillation techniques, and efficient training methods. With these theoretical concepts defined, the following chapter describes the specific methodology and system architecture designed to implement Symbolic Chain-of-Thought Distillation.

3. Methodology

This chapter details the practical implementation of the research. It outlines the system requirements and architecture, describing the complete pipeline from teacher data generation and cleaning to the specific configurations used for student fine-tuning and evaluation.

3.1. System requirements

The objective of this system is to facilitate the distillation of reasoning capabilities from a large teacher model to a smaller student model. The following requirements define the necessary functionality and operational constraints to achieve this goal within the specified resource limits.

3.1.1. Functional requirements

The system must provide specific capabilities to support the distillation pipeline. Primarily, it shall generate a dataset of Chain-of-Thought (CoT) reasoning traces for the GSM8K training set using a teacher model. To enable downstream analysis of generation efficiency and quality, the system must persist metadata for each generated sample, including latency, token counts, correctness status, and estimated cost. Furthermore, the system shall support fine-tuning of a student model in two distinct modes: one that learns to generate full reasoning traces (SCoTD) and one that learns to generate only the final answer (Label-only). Finally, the system shall evaluate the performance of base, student, and teacher models on the GSM8K test set using a strict numeric exact-match metric.

3.1.2. Non-functional requirements

To ensure the system is practical and reliable, several non-functional requirements must be met. The data generation module must operate asynchronously to maximize throughput while handling API rate limits and timeouts effectively. Observability is critical so system must log detailed metrics during generation to allow for real-time monitoring of costs and progress. Resource efficiency is a key constraint, meaning the training and evaluation pipelines must be optimized to run within the memory limits of a single 24GB VRAM GPU. Additionally, reproducibility is essential, requiring that all experiments use fixed random seeds, versioned datasets, and deterministic configuration files.

3.1.3. Constraints

The project operates under specific constraints that influence design and implementation choices. The primary hardware constraint is the limitation to a single GPU environment, which necessitates the use of parameter-efficient fine-tuning methods like LoRA and quantization. The budget for API usage is capped at 200 PLN, requiring a cost-aware approach to data generation. Additionally, the teacher model is accessed via an external API, introducing latency and potential availability issues. Finally, the context window is limited, meaning excessive rationales must be pruned or truncated without losing the final answer line.

3.1.4. Success criteria

The project will be considered successful if the distilled student model achieves an accuracy improvement of at least 5 percentage points over the base CoT-prompted model on the GSM8K test set. Additionally, the cleaned dataset must be of sufficient size (tens of thousands of rationale-answer pairs) to enable measurable gains without exceeding resource limits. All training and benchmarking runs must complete within the available GPU memory and budget envelope.

3.1.5. Risks and mitigation

Several risks have been identified that could impact the success of the project. There is a risk that the student model may overfit to the limited training subset. This is mitigated by generating multiple diverse reasoning paths per question and applying regularization during training. The student model may fail to adhere to the strict output format required for evaluation. This is addressed by using consistent prompt templates and enforcing strict parsing logic. Incorrect reasoning traces from the teacher could degrade student performance. A strict filtering step is implemented to remove any samples where the final answer does not match the ground truth. Uncontrolled API usage could exceed the budget. This is mitigated by monitoring spend and setting account balance limits.

3.2. Architecture

The system is designed as a modular pipeline that goes from data generation to model training and evaluation. It is an offline teacher-to-student pipeline that produces a cleaned CoT dataset and trains a 3B-class student with PEFT, followed by evaluation under strict numeric exact match. The high-level structure of this pipeline is depicted in Figure 3.1.

3.2.1. Components and data flow

The architecture comprises several key components: a Teacher Generator (an async OpenRouter client), data analysis and processing scripts for cleaning and deduplication, a training module utilizing QLoRA, and an evaluation module for benchmarking.

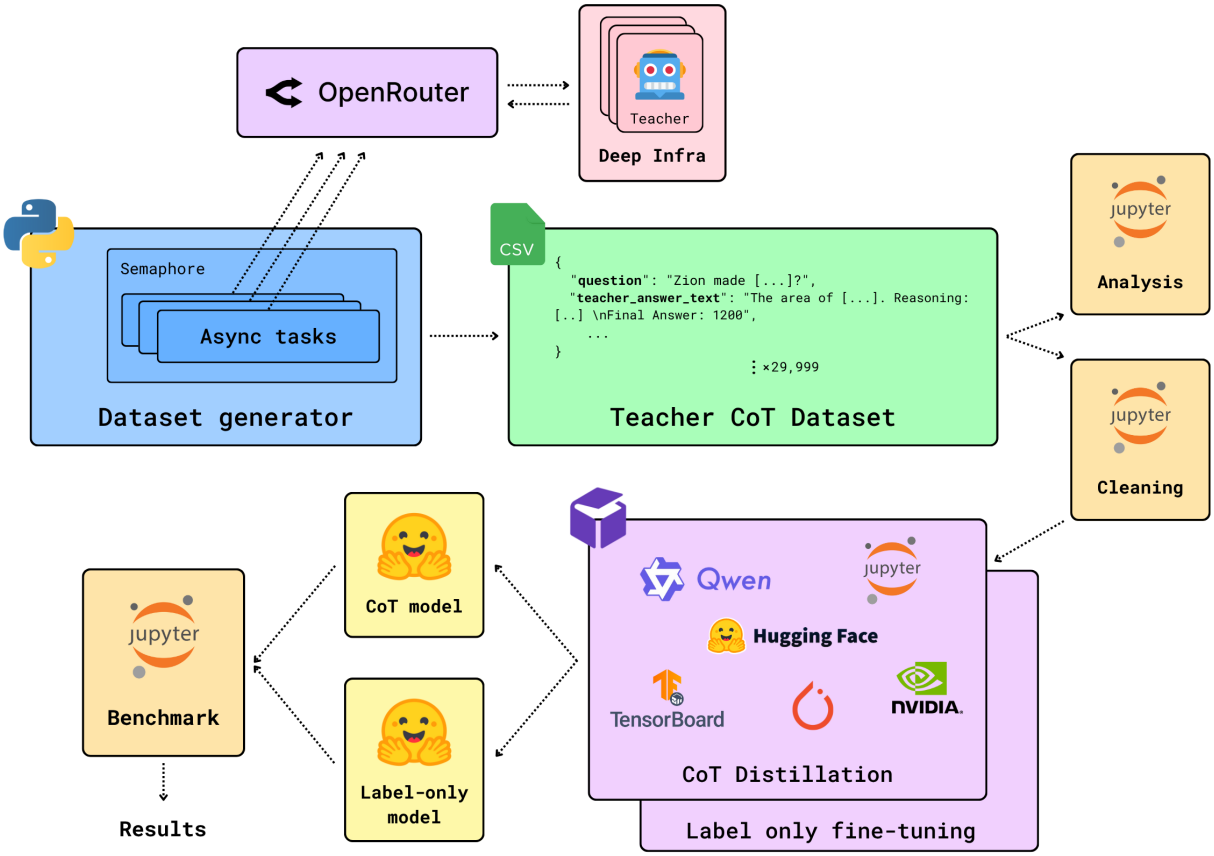


Figure 3.1. End-to-end SCoTD pipeline.

The data flows through the system in a linear sequence. First, the system loads the GSM8K train/test splits and selects the first ∼1,000 training questions. Then, the Teacher Generator oversamples ∼30 CoTs per question asynchronously, writing the results to a raw JSONL file. The raw data undergoes correctness filtering, rationale deduplication, and length trimming to produce a cleaned JSONL dataset. The cleaned dataset is projected to a minimal supervised learning schema. The student model is fine-tuned in two modes (SCoTD and label-only). Finally, the models are benchmarked using strict numeric exact-match evaluation.

3.3. Teacher CoT dataset generation

This stage produces a cleaned high-quality rationale+answer corpus from a strong reasoning model.

3.3.1. Subset selection and async generation

For cost control, only the first ∼1,000 GSM8K train questions are used. Each question targets 30 samples to ensure rationale diversity. An async Python client (HTTPX) calls the OpenRouter API for the `deepseek-r1-distill-qwen-32b` model. Concurrency is capped at 25 via a semaphore to balance throughput with rate limits, with a request timeout set to 120 seconds. Each request includes a system prompt specifying the reasoning format and a user prompt with few-shot exemplars.

3.3.2. Formatting and correctness

The teacher is required to output a structured response containing reasoning steps followed by a final answer line (e.g., "Final Answer: <number>"). The system parses this output to extract the numeric answer. Correctness is determined by a strict numeric equality check against the gold answer from the GSM8K dataset. Any parsing failure or missing final line marks the sample as invalid.

3.3.3. Persistance and progress monitoring

Each raw sample is appended to a JSONL file with metadata including the question ID, sample ID, full text, correctness status, token usage, and latency. The process is resumable; existing samples are skipped on subsequent runs. A live progress bar displays real-time metrics such as cost (estimated at $0.27 per million tokens), cumulative token usage, and running accuracy. Failures (e.g., timeouts, API errors) are tracked but not written to the dataset.

3.4. Cleaning

This stage removes incorrect, duplicate, and extreme-length samples to yield a high-signal supervision set. The cleaning process loads the raw JSONL data and applies a series of filters. First, a correctness filter retains only rows where the teacher's answer matches the gold standard, preventing the student from learning incorrect reasoning. Second, exact rationale deduplication removes duplicate reasoning paths to avoid overweighting specific patterns. Finally, token-length outlier removal discards samples exceeding the 99th percentile of length (threshold 1392 tokens) to control memory usage and sequence length. The resulting dataset contains 24,952 cleaned samples with 100% teacher correctness.

3.5. Preprocessing

The preprocessing stage transforms the cleaned corpus into the minimal supervision schema consumed by training scripts. It selects only the necessary fields: question, gold answer text and number, and the teacher's rationale and answer. This minimal schema reduces I/O and memory footprint while preserving all information needed for training and evaluation.

3.6. Prompt templates

Fixed prompt templates are used across generation, training, and benchmarking to ensure consistency. These templates are versioned as plain text files. The **CoT system prompt** instructs the model to provide concise multi-step reasoning followed by a final answer line:

```
You are a rigorous yet concise math solver. Solve problem
with a brief and correct chain-of-thought followed by a
```

```
single final answer line.

Output format (strict):
Reasoning:
- step 1
- step 2
- step 3
[up to 8 short steps total]
Final Answer: <number>

Rules:
- 2-8 short reasoning steps.
- Do not restate the question or add commentary.
- Show essential arithmetic explicitly (e.g., 3x7=21).
- No extra sections, no citations.
- Final line must be exactly: Final Answer: <number>
- Stop after the final answer line.
```

The **CoT user prompt** provides few-shot exemplars demonstrating this format with example problems and their step-by-step solutions. Similarly, the **Label-only system prompt** constrains the output to a single final answer line without reasoning:

```
You are a rigorous and precise math solver. Output the
correct solution as a single final answer line:
"Final Answer: <number>"

Output format (strict):
Final Answer: <number>

Rules:
- Do not restate the question or add commentary or reasoning.
- No extra sections, no citations.
- Output must be exactly one line: Final Answer: <number>
- Do not add units to the final number.
- Stop immediately after the answer line.
```

The formatting contract requires the final line to start exactly with “Final Answer:” followed by the number, with no trailing text.

3.7. Training

The training process is designed to fine-tune a single 3B decoder-only model (Qwen/Qwen2.5-3B) using QLoRA. This approach enables efficient training on consumer hardware while preserving the reasoning capabilities of the base model.

3.7.1. Model configuration and data

The base model is loaded with 4-bit NF4 quantization and double quantization to minimize memory usage, while computation is performed in bfloat16 precision. LoRA adapters are attached to all linear projection layers (attention and MLP) with a rank of 16 and alpha of 32. The dataset is split into training (80%) and validation (20%) sets using a fixed seed to ensure reproducibility and prevent data leakage.

3.7.2. Training procedure

Training utilizes the HuggingFace Trainer with the `paged_adamw_8bit` optimizer. A linear learning rate scheduler is employed with a warmup period. Sequences are constructed by concatenating the prompt and the target answer, with the loss calculated only on the target tokens. The maximum sequence length is set to 2048 tokens. If truncation is necessary, the prompt is trimmed from the left to preserve the full answer.

Two distinct training modes are executed. The **SCoTD mode** trains the model to generate the full teacher rationale and final answer, using a learning rate of 2×10^{-5} for 10 epochs, a per-device batch size of 8, and 2 gradient accumulation steps. The **Label-only mode** trains the model to generate only the final answer line, using a learning rate of 1×10^{-5} for 20 epochs, a batch size of 16, and 1 accumulation step.

3.7.3. Export

After training, the best LoRA checkpoint (selected based on validation loss) is merged with the base model for inference. Deterministic greedy decoding is used for sanity checks to ensure the model adheres to the expected output format.

3.8. Benchmarking

Benchmarking evaluates the base model and all student checkpoints on the GSM8K test split. The primary objective is to measure the exact-match accuracy of the final numeric answer.

3.8.1. Evaluation pipeline

The evaluation process loads the GSM8K test set and extracts the gold numeric answers. For each model (base, SCoTD, label-only), inference is performed using deterministic greedy decoding with a

maximum token limit sufficient for long rationales. The models are loaded with the same 4-bit quantization used during training. Inference is batched to optimize throughput, using a batch size of 16 for CoT generation and 128 for label-only generation.

3.8.2. Scoring

The generated outputs are parsed to extract the final answer using strict rules. The parser searches for the "Final Answer:" line and extracts the last numeric token. Any deviation from this format, including missing lines or malformed numbers, results in a parsing error and is counted as an incorrect answer. Accuracy is calculated as the fraction of predictions where the parsed number exactly matches the gold number.

3.9. Environment setup

The experiments were conducted on a single RunPod instance equipped with an NVIDIA RTX 4090 GPU (24GB VRAM) and 64GB of system RAM. The software environment was based on Python 3.11, utilizing PyTorch 2.8.0 and CUDA 12.9. The `uv` package manager was used for dependency management, and the environment was configured to disable tokenizer parallelism to avoid deadlocks.

In summary, this chapter has defined the experimental setup, including the data processing pipeline, training configurations, and evaluation metrics. The next chapter presents the empirical results obtained from this system, quantifying the performance of the distilled models compared to the baselines.

4. Results

This chapter reports empirical outcomes of symbolic chain-of-thought distillation (SCoTD) and label-only finetuning on GSM8K. All metrics use strict numeric exact-match evaluation under greedy decoding. No self-consistency sampling or answer normalization was applied.

4.1. Evaluation setup recap

The evaluation framework relies on the `Qwen/Qwen2.5-3B` base model, which serves as the starting point for all experiments. Two student models were trained using LoRA adapters on cleaned teacher data: the SCoTD student, which learns to generate both the rationale and the answer, and the Label-only student, which is trained solely on the final answer line. The teacher model, `deepseek/deepseek-r1-distill-qwen-32b`, was accessed via API to generate the training data.

For all benchmarks, decoding was performed greedily (`do_sample=false`) to ensure deterministic results. The primary metric is accuracy, defined as the ratio of correct numeric parses to the total number of questions in the GSM8K test set (1,319 problems). The parsing logic is strict, requiring the output to contain a line starting with `Final Answer:` followed immediately by a single numeric token. Any deviation from this format is counted as an incorrect response.

Model	Accuracy (%)
Teacher (API)	93.61
Student SCoTD (best checkpoint)	78.54
Base (CoT prompting)	70.51
Student label-only (best checkpoint)	15.01
Base (label-only prompting)	10.99

Table 4.1. GSM8K test set accuracies (strict numeric exact-match, greedy decoding).

As shown in Table 4.1, SCoTD yields a +8.03 percentage-point gain over the base CoT baseline, while label-only supervision gives a +4.02 point gain over the label-only baseline under strict formatting.

4.2. Checkpointing

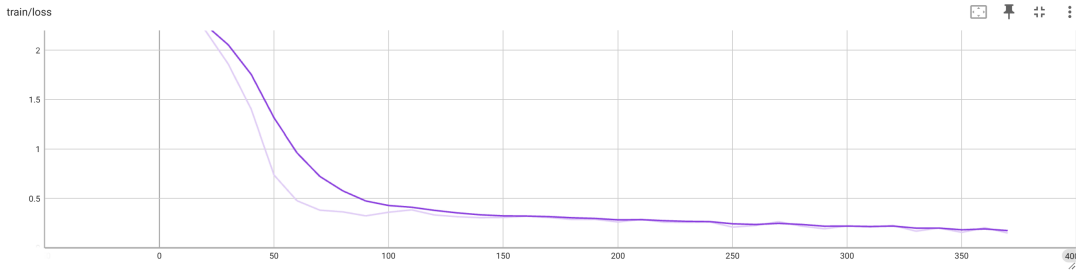
The training process involved saving model checkpoints at regular intervals to monitor performance and identify the optimal stopping point. For the SCoTD model, checkpoints were saved every 200 training steps. The best accuracy was observed at step 1,400, reaching 78.54%. Checkpoints saved after this point exhibited slight stagnation, with no further gains in accuracy despite a continued decrease in training loss. This pattern indicates diminishing returns and potential mild overfitting to the rationale style rather than an improvement in numeric robustness.

In contrast, the Label-only model was saved more frequently, every 50 steps, due to its faster convergence. The best accuracy was recorded early at step 200 (15.01%). Subsequent checkpoints showed a degradation or plateau in performance, which correlated with a rising validation loss after an early minimum.

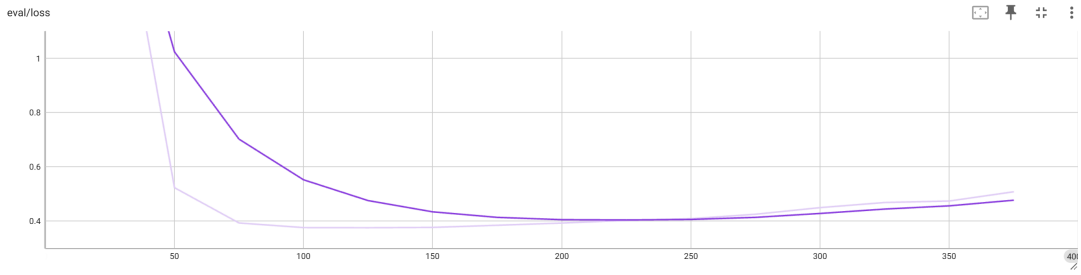
4.3. Training

This section presents the training dynamics of both models, examining the evolution of loss functions and optimization metrics throughout the training process. The label-only and SCoTD models exhibit distinct behaviors that reflect the nature of their respective supervision signals.

4.3.1. Label-only model

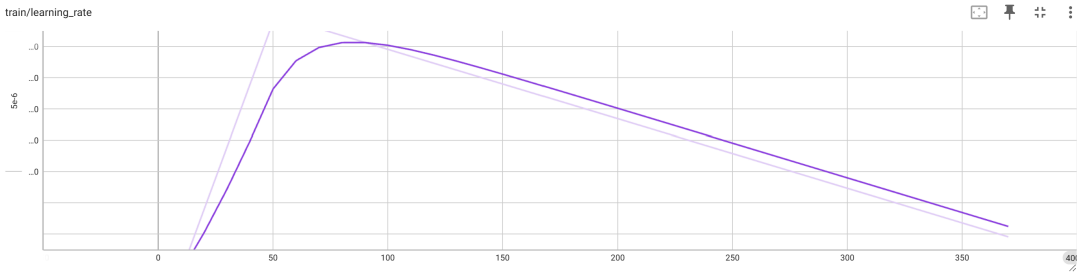


(a) Train loss

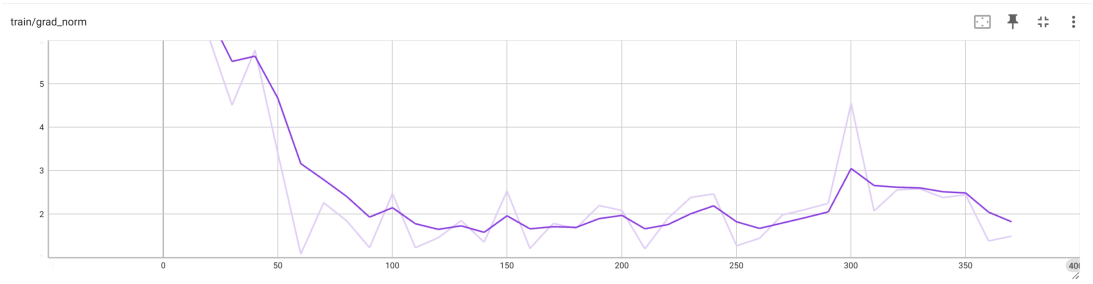


(b) Eval loss

Figure 4.1. Label-only losses: rapid early train loss compression; eval loss reaches minimum near 180–220 steps then rises.



(a) Learning rate



(b) Grad norm

Figure 4.2. Label-only optimization dynamics: warmup then linear LR decay; early high gradient norms (> 6) collapse to a stable 1.5–2.5.

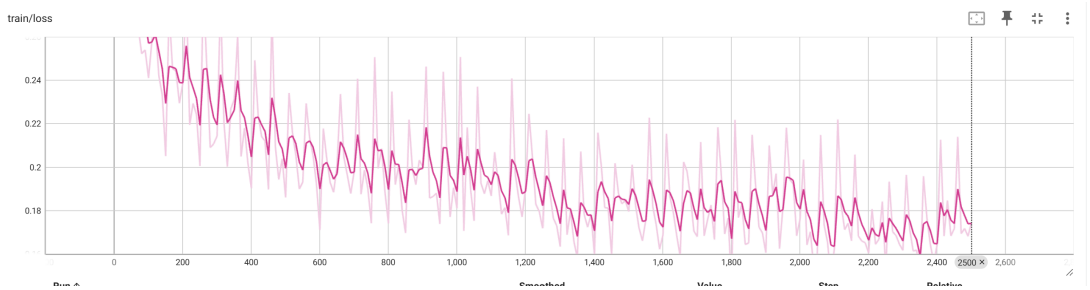
The training dynamics of the label-only model are characterized by rapid optimization. As illustrated in Figure 4.1a, the training loss drops by more than 75% within the first 120 steps. This swift adaptation is mirrored in the evaluation loss (Figure 4.1b), which reaches a minimum near steps 180–220. This window aligns with the peak accuracy of 15.01%. The subsequent rise in evaluation loss indicates overfitting, likely to specific formatting or token patterns rather than underlying reasoning. Additionally, the gradient norms (Figure 4.2b) show a large initial adaptation followed by a collapse to a stable range, reflecting the limited sequence length of the targets.

4.3.2. SCoTD model

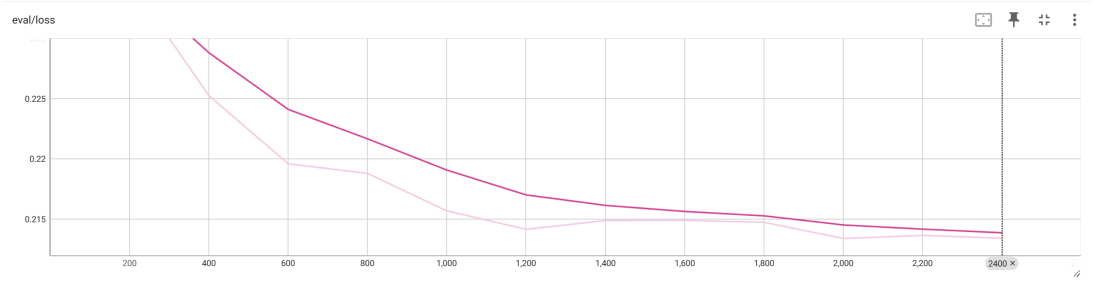
The SCoTD model exhibits more stable optimization behavior. Figure 4.4b shows an absence of large gradient spikes, likely due to the longer sequences distributing updates more evenly. Unlike the label-only model, the evaluation loss for SCoTD continues to improve past the point of best accuracy (Figure 4.3b). This implies that accuracy saturation occurs before the token-level optimum is reached. The gradient norms remain within a constrained range, suggesting consistent convergence and indicating that the diversity of rationales did not destabilize the training process.

4.3.3. Comparison

The two training approaches exhibit distinct behaviors that reflect their underlying supervision signals. The label-only model achieves faster early loss reduction but suffers from an earlier generalization

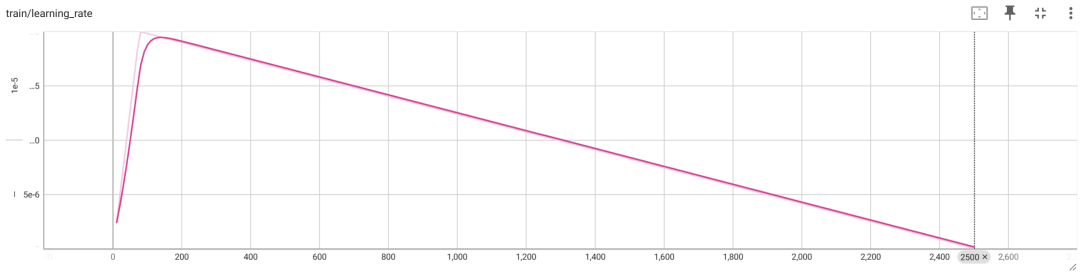


(a) Train loss

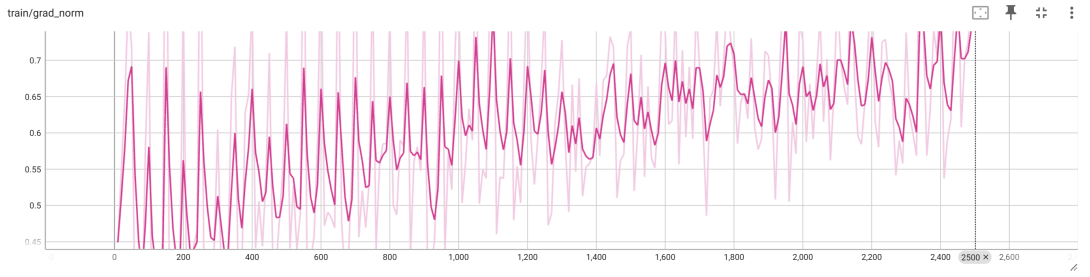


(b) Eval loss

Figure 4.3. SCoTD losses: steady train loss descent; eval loss monotonically declines without early reversal.



(a) Learning rate



(b) Grad norm

Figure 4.4. SCoTD dynamics: short warmup then linear LR decay; grad norms in a tight 0.45–0.75 band with slight late drift upward.

peak and subsequent overfitting, consistent with its short target sequences and limited supervision richness. In contrast, SCoTD demonstrates sustained improvement in evaluation loss throughout training,

where the accuracy plateau likely reflects a reasoning ceiling rather than overfitting. The gradient behavior also differs significantly: the high initial norms of the label-only model contrast with the moderate, steady norms of SCoTD, indicating incremental adaptation when learning longer rationales. These observations suggest that rationale supervision provides a more stable optimization.

4.4. Output length and latency distributions

The distribution of reasoning steps and generation latencies is shown in Figure 4.5.

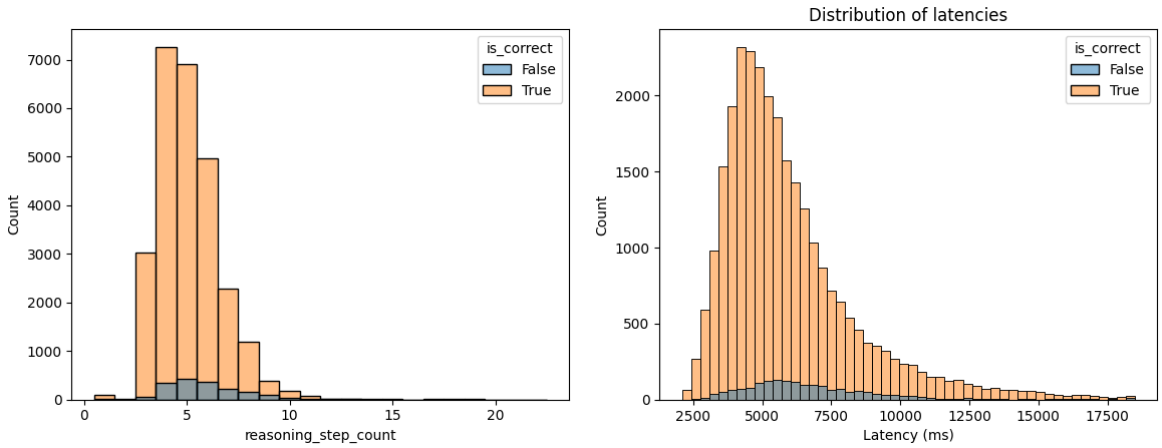


Figure 4.5. Left: reasoning step count (mode 5, most in [4,7], rare long tails > 12). Right: latency distribution (median $\sim$5.4 s, long right tail to > 18 s).

Figure 4.6 illustrates the per-query cost distribution and the length of the generated answers in characters and tokens.

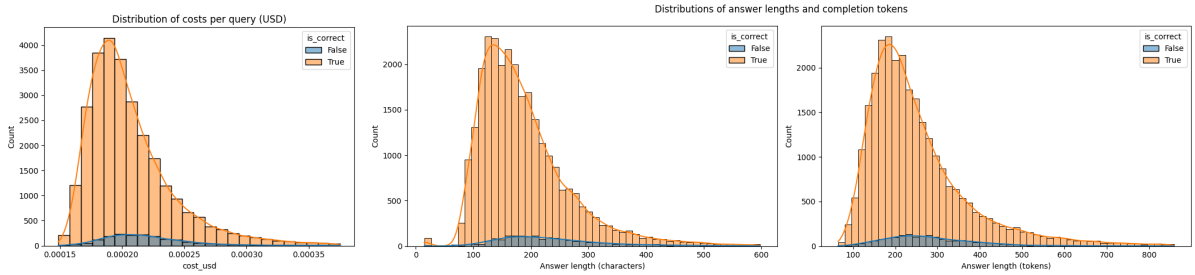


Figure 4.6. Left: cost per query (right-skew, peak near \$0.00019–0.00021). Right: answer length distributions (characters and tokens; token mode $\sim$180, tail > 800).

Analysis of the reasoning step counts shows that 90% of the generated samples fall within the range of 4 to 7 steps. Interestingly, incorrect samples are slightly over-represented in the 4–5 step range, suggesting that shorter reasoning chains may correlate with lower accuracy. Regarding latency, the interquartile range spans from approximately 4.3 seconds to 7.1 seconds, with a heavy tail that aligns with the longest completions.

The cost distribution approximates a log-normal shape, with a long tail extending beyond $0.00030 that consists mainly of verbose rationales. In terms of length, the median character count is around 150 and the median token count is 225. Incorrect answers tend to be mildly shorter, which may indicate information insufficiency in those responses.

4.5. Dataset filtering outcomes

The data generation phase produced a total of 30,000 raw samples, derived from 1,000 training questions each queried 30 times. The teacher model demonstrated a high level of numeric correctness on this raw set, achieving 93.75% (28,125 correct samples).

To ensure high-quality supervision, the raw data underwent a rigorous cleaning process. First, 1,875 incorrect answers were removed to prevent the student from learning faulty reasoning. Next, 2,917 exact duplicate rationales were eliminated to avoid bias. Finally, length outliers exceeding the 99th percentile (approximately 1,392 tokens) were discarded, affecting about 300 samples. This process resulted in a final dataset of 24,952 cleaned samples, all of which are numerically correct.

4.6. Cost analysis

The total cost for teacher generation via the API amounted to $6.33. The average cost per raw query was $0.00021, with a median of $0.00020 and a standard deviation of $0.000062. When amortized over the retained high-quality data, the cost per cleaned sample was approximately $0.000254. The computational cost for training and evaluation on the RunPod infrastructure was $18.61.

This chapter has presented the quantitative findings, demonstrating the accuracy improvements achieved by the SCoTD approach and the training dynamics observed. The following chapter interprets these results, discussing their implications, the impact of dataset filtering, and the limitations of the study.

5. Discussion

This chapter provides an in-depth analysis of the results reported in the previous chapter. It interprets the quantitative outcomes, examines the influence of dataset quality and formatting constraints, and contextualizes the findings within the broader landscape of current research and the study's limitations.

5.1. Interpretation of quantitative outcomes

The distilled student (78.54% accuracy) improves over the base CoT-prompted model (70.51%) by +8.03 percentage points under strict greedy evaluation. This gain indicates that symbolic chain-of-thought distillation (SCoTD) successfully transfers structured multi-step reasoning patterns rather than merely formatting habits. The teacher at 93.61% establishes remaining headroom: even after distillation the student closes only about 35% of the gap to the teacher (absolute difference 23.10 points). Label-only fine-tuning produces a smaller +4.02 point improvement (15.01% vs 10.99%), reflecting that (i) training only the final answer line provides limited signal for intermediate decomposition, and (ii) strict formatting magnifies penalties for subtle deviations.

5.2. Role of dataset filtering

The cleaned dataset (24,952 samples, 100% numerically correct) removes incorrect answers, near-duplicate rationales, and extreme-length outliers. Eliminating label noise prevents the student from learning wrong numeric terminations; rationale deduplication reduces gradient redundancy and broadens reasoning variety; length capping protects memory.

5.3. Training dynamics and generalization

Label-only mode shows rapid early optimization, an early eval loss floor, and overfitting signs after approximately 200 steps, consistent with short targets and limited supervision richness. SCoTD exhibits slower but steadier improvement: eval loss continues decreasing past the best accuracy checkpoint, implying a saturation of accuracy before full convergence of token-level modeling. This mismatch indicates that incremental gains in reproducing teacher phrasing beyond a certain point do not translate into additional numeric correctness under greedy decoding.

5.4. Formatting sensitivity

Strict parsing (exact “Final Answer: <number>”) penalizes:

– Additional commentary after the numeric token.

– Missing prefix or variant capitalization.

– Multiple numeric candidates on the line.

The observed low label-only accuracy relative to intuitive difficulty of many questions suggests a good fraction of outputs may contain correct numbers with minor formatting deviations (not separately quantified—future work should measure the formatting error rate).

5.5. Cost and efficiency implications

Teacher generation cost $6.33 produced a high-signal corpus. Oversampling (30 CoTs per question) on a subset of GSM8K, yield a mean per-sample API cost of only $0.00021. Two training runs on a single 24 GB GPU incurred $18.61 in total compute, demonstrating that QLoRA fine-tuning can produce measurable reasoning gains on commodity hardware without large-scale infrastructure.

5.6. Limitations and threats to validity

Several limitations constrain the generalizability of these findings. The study relies on a single model architecture (Qwen2.5-3B) and a single dataset (GSM8K), which prevents a broader assessment of how SCoTD scales or applies to domains requiring more complex symbolic manipulation or formal proofs. Additionally, the evaluation did not employ self-consistency decoding [3], leaving potential additive gains unmeasured. A significant threat to validity is the possibility of data contamination, as the teacher model’s pre-training data may have included the GSM8K test set, potentially inflating the quality of the generated supervision. Furthermore, the strict formatting requirements of the evaluation parser likely underestimate the true numeric capability of the models. This is particularly relevant in the label-only setting, where minor syntactic deviations result in penalties that may not reflect underlying reasoning failures.

5.7. Comparison to literature

The results obtained in this study align with work on symbolic chain-of-thought distillation by Li et al. [21], which states that small models can learn reasoning patterns from larger teachers. While early research on CoT prompting suggested that reasoning capabilities emerge primarily in models larger than 10B parameters [2], our findings confirm that a 3B parameter model can indeed improve its reasoning accuracy (from 70.51% to 78.54%) when explicitly trained on reasoning traces. This contrasts with traditional distillation methods focused on logit approximation [16, 19], demonstrating that sequence-level

supervised fine-tuning on rationales is a viable alternative when access to teacher logits is restricted (e.g., API-based teachers). Furthermore, the performance gap between the student and the teacher (93.61%) is consistent with the capacity gap expected between 3B and 32B parameter models, suggesting that while distillation bridges the gap, model scale remains a dominant factor in absolute performance.

5.8. Implications and future directions

Despite the identified limitations, the study has clear implications for deploying efficient reasoning models. It demonstrates that high-quality, filtered CoT data enables small models to perform multi-step arithmetic reasoning without increasing inference latency. The results show that parameter-efficient fine-tuning (LoRA + QLoRA) is sufficient to encode these capabilities on consumer-grade hardware, and that rationale supervision yields better returns than answer-only supervision for math problems under greedy decoding. Future work should extend this approach to multiple model architectures and datasets to evaluate cross-domain generalization. Further research is also needed to quantify the trade-off between formatting errors and numeric accuracy, to assess the marginal gains of self-consistency sampling for distilled students, and to explore whether distillation loss functions could provide deeper knowledge transfer than standard language modeling loss.

6. Conclusion

The thesis investigated whether symbolic chain-of-thought distillation (SCoTD) can improve a 3B-class small language model's math reasoning accuracy on GSM8K under strict greedy evaluation. It further compared rationale+answer supervision to answer-only (label-only) fine-tuning within a constrained single-GPU environment.

To address the specific research questions posed at the beginning of this thesis, the experimental results provide clear answers. Regarding **RQ1**, SCoTD improved the student from 70.51% (base with CoT prompting) to 78.54% (+8.03 points). Thus, supervised rationale transfer is effective for multi-step arithmetic in a 3B model under resource constraints. Regarding **RQ2**, label-only fine-tuning improved from 10.99% to 15.01% (+4.02 points) but remained far below SCoTD. This demonstrates that intermediate supervision is substantially more informative than final-answer-only supervision for this task and evaluation method.

The research conducted in this thesis led to the following key observations:

- Distilling structured teacher reasoning traces yields significant accuracy gains without increased inference-time cost.

- Rationale diversity (30 generations/question) combined with correctness filtering and deduplication produces a high-signal training set.

- Parameter-efficient fine-tuning provides adequate adaptation capacity for reasoning improvements in small models under tight resource constraints.

- Strict formatting multiplies penalties for minor output deviations, especially for label-only training; evaluation design noticeably affects reported performance.

However, the study has limitations. The reliance on a single dataset and model, the absence of self-consistency decoding, and the lack of formatting error analysis limit the generalizability and depth of insights.

In conclusion, symbolic chain-of-thought distillation offers a cost-efficient method to elevate small model reasoning without reliance on large teacher inference at deployment time. Under constrained hardware, combining high-quality rationale curation with parameter-efficient fine-tuning yields meaningful accuracy gains and provides a reproducible path toward more capable small-scale reasoning systems.

6.1. Future work

Future work should address the identified gaps by extending the methodology to additional reasoning benchmarks, model sizes, and architectures. It would be beneficial to analyze the influence of formatting errors on evaluation, experiment with self-consistency decoding to measure potential performance ceilings, and test different distillation loss functions to further optimize knowledge transfer.

Bibliography

[1] Karl Cobbe et al. „*Training Verifiers to Solve Math Word Problems*". 2021. arXiv: *2110.14168* [cs.LG].

[2] Jason Wei et al. „*Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*". In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo et al. Vol. 35. Curran Associates, Inc., 2022, pp. 24824–24837.

[3] Xuezhi Wang et al. „*Self-Consistency Improves Chain of Thought Reasoning in Language Models*". 2023. arXiv: *2203.11171* [cs.CL].

[4] Edward J Hu et al. „*LoRA: Low-Rank Adaptation of Large Language Models*". In: *International Conference on Learning Representations*. 2022.

[5] Tim Dettmers et al. „*QLoRA: Efficient Finetuning of Quantized LLMs*". In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh et al. Vol. 36. Curran Associates, Inc., 2023, pp. 10088–10115.

[6] Thomas Wolf et al. „*HuggingFace's Transformers: State-of-the-art Natural Language Processing*". 2020. arXiv: *1910.03771* [cs.CL].

[7] Taku Kudo and John Richardson. „*SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing*". In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Ed. by Eduardo Blanco and Wei Lu. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 66–71. DOI: *10.18653/v1/D18-2012*.

[8] Rico Sennrich, Barry Haddow, and Alexandra Birch. „*Neural machine translation of rare words with subword units*". In: *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*. 2016, pp. 1715–1725.

[9] Qwen Team et al. „*Qwen2.5 Technical Report*". 2025. arXiv: *2412.15115* [cs.CL].

[10] DeepSeek-AI et al. „*DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*". 2025. arXiv: *2501.12948* [cs.CL].

[11] OpenRouter. „*OpenRouter: The Unified Inference for LLMs*". *https://openrouter.ai*. Accessed: 2025-09-14. 2023.

[12] Ashish Vaswani et al. „*Attention is All you Need*". In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017.

[13] Jared Kaplan et al. „*Scaling Laws for Neural Language Models*". 2020. arXiv: *2001.08361* [cs.LG].

[14] Tom Brown et al. „*Language Models are Few-Shot Learners*". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle et al. Vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.

[15] Long Ouyang et al. „*Training language models to follow instructions with human feedback*". In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo et al. Vol. 35. Curran Associates, Inc., 2022, pp. 27730–27744.

[16] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. „*Distilling the Knowledge in a Neural Network*". 2015. arXiv: *1503.02531* [cs.CL].

[17] Tommaso Furlanello et al. „*Born Again Neural Networks*". In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Jennifer Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, Oct. 2018, pp. 1607–1616.

[18] Qizhe Xie et al. „*Self-Training With Noisy Student Improves ImageNet Classification*". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2020.

[19] Jianping Gou et al. „*Knowledge Distillation: A Survey*". In: *International Journal of Computer Vision* 129.6 (2021), pp. 1789–1819. DOI: *10.1007/s11263-021-01453-z*.

[20] Yoon Kim and Alexander M Rush. „*Sequence-level knowledge distillation*". In: *Proceedings of the 2016 conference on empirical methods in natural language processing*. 2016, pp. 1317–1327.

[21] Liunian Harold Li et al. „*Symbolic Chain-of-Thought Distillation: Small Models Can Also "Think" Step-by-Step*". 2024. arXiv: *2306.14050* [cs.CL].

[22] Neil Houlsby et al. „*Parameter-Efficient Transfer Learning for NLP*". In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, Sept. 2019, pp. 2790–2799.

[23] Brian Lester, Rami Al-Rfou, and Noah Constant. „*The Power of Scale for Parameter-Efficient Prompt Tuning*". In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Ed. by Marie-Francine Moens et al. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 3045–3059. DOI: *10.18653/v1/2021.emnlp-main.243*.

[24] Xiang Lisa Li and Percy Liang. „*Prefix-Tuning: Optimizing Continuous Prompts for Generation*". In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Online: Association for Computational Linguistics, Aug. 2021, pp. 4582–4597. DOI: *10.18653/v1/2021.acl-long.353*.

[25] „*bitsandbytes*". *https://github.com/bitsandbytes-foundation/bitsandbytes*.

[26] NVIDIA Corporation. „*QLoRA Parameter-Efficient Fine-Tuning - NVIDIA NeMo Documentation*". 2024. URL: *https://docs.nvidia.com/nemo-framework/user-guide/24.09/sft_peft/qlora.html*.

[27] Quentin Lhoest et al. „*Datasets: A Community Library for Natural Language Processing*". In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Ed. by Heike Adel and Shuming Shi. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 175–184. DOI: *10.18653/v1/2021.emnlp-demo.21*.